Advanced Data Structures and Algorithm Analysis

Laboratory Project Report · Project 1

Performance Comparison and Analysis of Search Trees

Yixiang Chen **Runhui Lin**

Zhejiang University

Date: 2026-10-19

Abstract

Search trees serve as fundamental building blocks for dynamic ordered dictionaries in both in-memory runtime libraries and external-storage indexing engines. In this project, we implement five representative search tree structures in C: the **unbalanced Binary Search Tree**, **AVL Tree**, **Splay Tree**, **Red-Black Tree**, and order-4 **B$^+$ Tree**. To evaluate their **asymptotic complexity** and structural resilience through **empirical benchmarking**, we test all five trees over datasets of N distinct integers ranging from 1,000 to 50,000 under three canonical workloads: **increasing-order insertion and increasing-order deletion**, **increasing-order insertion and reverse-order deletion**, and **random-order insertion and random-order deletion**. At $N = 50{,}000$, empirical measurements confirm that while an unbalanced BST achieves the lowest constant-factor overhead on randomized inputs with 5.52 ms insertion and 5.36 ms deletion, it suffers catastrophic $\Theta(N^2)$ degeneration on monotone sequences, taking 1.89 s for insertion and 1.78 s for reverse deletion. Conversely, Splay Trees exploit sequential spatial locality to achieve $\Theta(N)$ total time of 0.78 ms for sorted insertions, Red-Black Trees offer the most consistent all-around performance across adversarial and random workloads through bounded rotation counts, AVL Trees trade higher rotation frequency for stricter height bounds, and B$^+$ Trees incur higher CPU array-shifting overhead per update.

Keywords: Binary search tree; AVL tree; splay tree; red-black tree; B$^+$ tree; empirical benchmarking; asymptotic complexity.

Fall–Winter 2026

Contents

1 Introduction **1**
- 1.1 Problem Description . . . 1
- 1.2 Purpose of This Report . . . 1

2 Data Structure and Algorithm Specification **2**
- 2.1 Overview of Node Specifications . . . 2
- 2.2 Unbalanced Binary Search Tree . . . 2
- 2.3 AVL Tree . . . 2
- 2.4 Splay Tree . . . 3
- 2.5 Red-Black Tree . . . 3
- 2.6 B+ Tree . . . 4

3 Testing Results **5**
- 3.1 Test Suite and Structural Verification . . . 5
- 3.2 Performance Benchmark Setup and Runtime Table . . . 6
- 3.3 Runtime Plots vs. Input Size . . . 6

4 Analysis and Comments **8**
- 4.1 Time and Space Complexity Analysis . . . 8
- 4.2 Workload-Specific Empirical Behavior . . . 8
  - 4.2.1 Asymmetry of Unbalanced BST Under Ordered vs. Reverse Deletion . . . 8
  - 4.2.2 Sequential Locality Advantage of Splay Trees . . . 9
  - 4.2.3 Trade-offs Among Balanced Trees under Random Workloads . . . 9
- 4.3 Further Possible Improvements . . . 10

Author List and Declaration **11**

References **11**

A Source Code **12**
- A.1 Unbalanced Binary Search Tree (`binaray_search_tree.h`) . . . 12
- A.2 AVL Tree (`AVL_tree.h`) . . . 14
- A.3 Splay Tree (`splay_tree.h`) . . . 16
- A.4 Red-Black Tree (`red_black_tree.h`) . . . 18
- A.5 Order-4 B+ Tree (`B_plus_tree.h`) . . . 23
- A.6 Performance Benchmark Driver (`testbench_time.c`) . . . 29

Chapter 1. Introduction

1.1 Problem Description

Maintaining a dynamic collection of ordered keys under frequent insertions, deletions, and lookups is a core problem in computer science. Although an ordinary Binary Search Tree supports these dictionary operations in $O(\log N)$ time on average when keys arrive in random permutations [3], it offers no worst-case structural guarantee. Under adversarial or naturally ordered arrival streams, an ordinary BST degenerates into a singly linked chain of height $N-1$, degrading individual updates to $\Theta(N)$ time and a sequence of N operations to $\Theta(N^2)$ time.

To overcome tree degeneration without sacrificing ordered traversal semantics, four major families of self-balancing and self-adjusting search trees are covered in the ADS course lectures [2]:

- **AVL Trees** enforce a strict local height-balance invariant $|\mathrm{BF}(v)| \leq 1$ at every node, guaranteeing worst-case $O(\log N)$ height via single and double rotations [2, 3].
- **Red-Black Trees** relax the height constraint into a two-color path invariant, bounding tree height by $2\log_2(N+1)$ while restricting structural fixes to at most $O(1)$ rotations per update [1, 2].
- **Splay Trees** eliminate explicit balance metadata altogether, moving every accessed node to the root via two-step *zig-zig* and *zig-zag* rotations to guarantee $O(\log N)$ amortized cost per operation [2, 3].
- **B$^+$ Trees** generalize binary trees into multi-way search structures where all keys reside in linked leaf nodes and internal nodes act as routing separators, optimizing node fan-out through node splitting and merging [2].

1.2 Purpose of This Report

The objective of this project is to implement all five search trees from scratch in C, verify their structural invariants, and systematically analyze and compare the performances of sequences of insertions and deletions across $N \in [10^3, 5 \times 10^4]$ distinct integers in three canonical ways:

1. **Ordered Insertion & Same-Order Deletion:** Insert N integers in increasing order $0, 1, \ldots, N-1$ and delete them in the same increasing order;
2. **Ordered Insertion & Reverse-Order Deletion:** Insert N integers in increasing order $0, 1, \ldots, N-1$ and delete them in the reverse decreasing order $N-1, N-2, \ldots, 0$;
3. **Random Insertion & Random Deletion:** Insert N integers in a uniform random order and delete them in an independent uniform random order.

By combining theoretical complexity proofs with detailed empirical runtime measurements, this report clarifies the practical trade-offs between height balance, color balance, self-adjustment, and multi-way branching.

Chapter 2. Data Structure and Algorithm Specification

2.1 Overview of Node Specifications

Table 1 summarizes the structural metadata, invariants, and memory layouts of the five implemented search trees.

Table 1: Comparison of structural representations and invariants across the five search trees.

Structure	Node Fields	Balancing Invariant
Unbalanced BST	`val`, `Left`, `Right`	None; tree shape depends entirely on insertion and deletion history.
AVL Tree	`val`, `height`, `BF`, `Left`, `Right`	Height difference $\lvert\mathrm{BF}(v)\rvert = \lvert h(v.\mathrm{Left}) - h(v.\mathrm{Right})\rvert \leq 1$ for every node v.
Splay Tree	`key`, `left`, `right`	No stored balance metadata; newly accessed, inserted, or split nodes are splayed to the root.
Red-Black Tree	`key`, `color`, `left`, `right`, `parent`	Root is black; red nodes have black children; every root-to-`NULL` path contains the same number of black nodes.
Order-4 B^+ Tree	`isLeaf`, `numKeys`, `keys[]`, `children[]`, `parent`, `next`	Internal nodes have 2 to 4 children and 1 to 3 separator keys; non-root leaves hold 2 to 3 keys and form a singly linked list.

2.2 Unbalanced Binary Search Tree

The standard search, insertion, and deletion rules of an unbalanced BST were already covered in *Fundamentals of Data Structures*, so we omit their basic specification here.

One thing worth mentioning in `binaray_search_tree.h` is that we implement both insertion and deletion in an **iterative** manner rather than a **recursive** one. When $N = 50{,}000$ sorted keys are inserted, the tree degenerates into a chain of depth N, causing **recursive** calls to overflow the system call stack around $N \approx 26{,}000$. Using explicit `parent` and `current` pointers in an **iterative** loop reduces auxiliary stack space to $O(1)$ and avoids stack overflow.

2.3 AVL Tree

In `AVL_tree.h`, each node stores its subtree `height`, with empty trees having height -1 and leaves height 0, together with the balance factor $\mathrm{BF}(v) = h(v.\mathrm{left}) - h(v.\mathrm{right})$. After a standard BST insertion or deletion, each ancestor v along the recursion return path updates its height and restores balance when $\lvert\mathrm{BF}(v)\rvert = 2$ according to four cases:

- **LL Case, $\mathrm{BF}(v) = +2$ and $\mathrm{BF}(v.\mathrm{left}) \geq 0$:** Perform a single right rotation at v to lift v.left as the new subtree root, as shown in Figure 1.
- **RR Case, $\mathrm{BF}(v) = -2$ and $\mathrm{BF}(v.\mathrm{right}) \leq 0$:** Perform a single left rotation at v to lift v.right as the new subtree root.
- **LR Case, $\mathrm{BF}(v) = +2$ and $\mathrm{BF}(v.\mathrm{left}) = -1$:** Perform a left rotation at v.left followed by a right rotation at v.
- **RL Case, $\mathrm{BF}(v) = -2$ and $\mathrm{BF}(v.\mathrm{right}) = +1$:** Perform a right rotation at v.right followed by a left rotation at v.

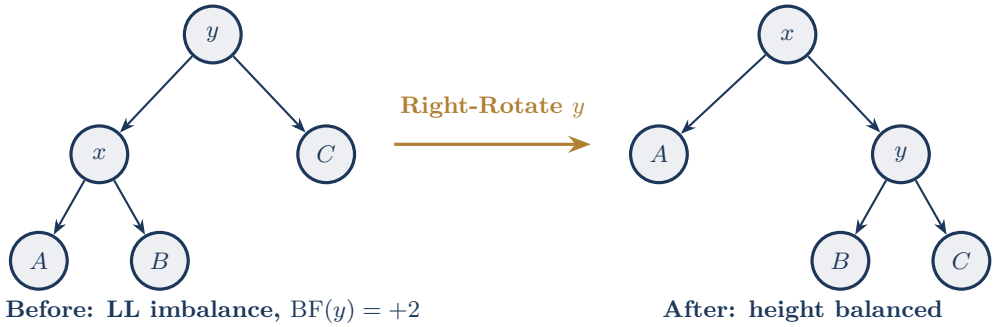


Figure 1: AVL LL single right rotation preserving in-order key sequence $A < x < B < y < C$.

Algorithm 1: Rebalance-AVL(v)

```
Update h(v) ← 1 + max(h(v.left), h(v.right)) and BF(v) ← h(v.left) − h(v.right).
if BF(v) > 1 then                                              ▷ Left-heavy subtree
    if BF(v.left) < 0 then v.left ← Left-Rotate(v.left)        ▷ Convert LR to LL
    return Right-Rotate(v)
if BF(v) < −1 then                                             ▷ Right-heavy subtree
    if BF(v.right) > 0 then v.right ← Right-Rotate(v.right)    ▷ Convert RL to RR
    return Left-Rotate(v)
return v
```

2.4 Splay Tree

In `splay_tree.h`, Splay(v, k) moves the accessed key or the last visited node x to the root via three rotation steps according to the relative positions of x, its parent p, and its grandparent g [2, 3]:

- **Zig Step:** When p is the tree root, perform a single rotation at p to lift x directly to the root.
- **Zig-Zig Step:** When x and p are both left children or both right children, rotate first at the grandparent g and then at the parent p, which halves the depth of nodes along the access path.
- **Zig-Zag Step:** When x is a left child and p is a right child, or vice versa, rotate first at p and then at g, matching the double rotation of an AVL tree.

Based on Splay(r, k), the insertion and deletion algorithms follow directly as shown below:

Algorithm 2: Splay-Insert(r, k) and Splay-Delete(r, k)

Insert: 1. **if** $r =$ NIL **then return** a new single-node tree with key k.
2. $r \leftarrow$ Splay(r, k); **if** r.key $= k$ **then return** r.
3. Create new root z with key k. **if** $k <$ r.key **then** set z.left $\leftarrow$ r.left, z.right $\leftarrow r$, and r.left $\leftarrow$ NIL; **else** symmetrically split r.right. **return** z.

Delete: 1. **if** $r =$ NIL **then return** NIL. Set $r \leftarrow$ Splay(r, k); **if** r.key $\neq k$ **then return** r.
2. **if** r.left $=$ NIL **then** $r' \leftarrow r$.right; **else** splay the maximum of the left subtree via $r' \leftarrow$ Splay$(r$.left$, k)$ and join r'.right $\leftarrow r$.right.
3. Free old root r and **return** r'.

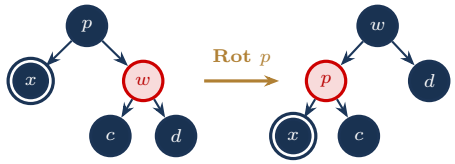
2.5 Red-Black Tree

In `red_black_tree.h`, each node stores a color $\in \{\text{RED}, \text{BLACK}\}$ and a `parent` pointer. Standard BST insertion colors the new leaf z RED; standard deletion replaces a two-child target with its in-order predecessor and removes node y. Violations of the red-black invariants are repaired iteratively as specified in Algorithm 3 and illustrated for all four double-black deletion cases in Figure 2 [1, 2]:

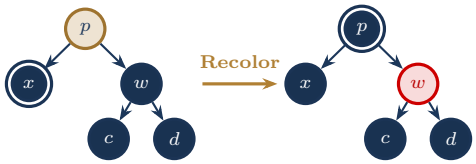
Algorithm 3: RB-Insert-Fixup(z) and RB-Delete-Fixup(x)

Insert Fix Let $g = p(p(z))$ be the grandparent and u be z's uncle. **while** color($p(z)$) = RED **do**:
- **Case 1,** color(u) = RED**:** Recolor $p(z), u \leftarrow$ BLACK, $g \leftarrow$ RED; advance $z \leftarrow g$.
- **Case 2,** u **is** BLACK **and** z **is an inner grandchild:** Rotate at $p(z)$ to reduce to Case 3.
- **Case 3,** u **is** BLACK **and** z **is an outer grandchild:** Recolor $p(z) \leftarrow$ BLACK, $g \leftarrow$ RED, rotate at g, and **break**. Finally set color(root) $\leftarrow$ BLACK.

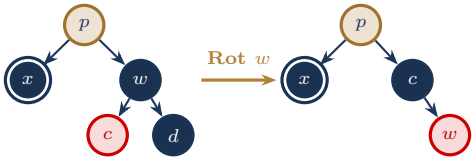
Delete Fix Let w be x's sibling. **while** $x \neq$ root and color(x) = BLACK **do**:
- **Case 1,** color(w) = RED**:** Recolor $w \leftarrow$ BLACK, $p(x) \leftarrow$ RED, rotate at $p(x)$, and update w.
- **Case 2,** w **and both nephews are** BLACK**:** Recolor $w \leftarrow$ RED; if $p(x)$ is RED, color $p(x) \leftarrow$ BLACK and **break**, else advance $x \leftarrow p(x)$.
- **Case 3,** w **is** BLACK**, inner nephew** RED**, outer** BLACK**:** Recolor w and inner nephew, then rotate at w to reduce to Case 4.
- **Case 4,** w **is** BLACK **and outer nephew is** RED**:** Set color(w) $\leftarrow$ color($p(x)$), color($p(x)$) $\leftarrow$ BLACK, outer nephew $\leftarrow$ BLACK, rotate at $p(x)$, and **break**.



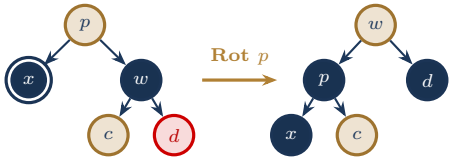
(a) Case 1: Red sibling w converts to Cases 2–4



(b) Case 2: Black w and black nephews c, d



(c) Case 3: Red inner nephew c converts to Case 4



(d) Case 4: Red outer nephew d restores balance

Figure 2: The four double-black deletion fix-up cases in a Red-Black Tree, where double-ringed nodes denote the extra black token and gold nodes denote arbitrary color.

2.6 B+ Tree

In `B_plus_tree.h` of order $M = 4$, all data keys reside in linked leaves with isLeaf = 1, while internal nodes store $k \in [1, 3]$ separator keys and $k + 1 \in [2, 4]$ children such that key$[i]$ = min(child$[i + 1]$) [2].

Algorithm 4: Order-4 B$^+$-Insert(T, k) and B$^+$-Delete(T, k)

Insert:
1. Descend to target leaf L; if $k \notin L$, insert k in sorted order and refresh ancestor separators.
2. **if** $|L| = M$ **then** split L into two leaves of $\lceil M/2 \rceil = 2$ keys, link L.next, and insert the new child pointer into $p(L)$.
3. Propagate splits upward: any internal node reaching $M + 1 = 5$ children splits into $\lceil 5/2 \rceil = 3$ and 2 children, creating a new root if the root splits.

Delete:
1. Locate leaf L and remove k. If $L = T$.root or $|L| \geq \lfloor M/2 \rfloor = 2$, refresh ancestor separators and **return**.
2. **Borrow: if** an adjacent sibling leaf S has more than 2 keys **then** transfer one boundary key from S to L and refresh ancestors.
3. **Merge: else** merge L with S, remove the redundant child pointer from $p(L)$, and recursively rebalance $p(L)$ when numKeys $< \lceil M/2 \rceil - 1 = 1$.

Chapter 3. Testing Results

3.1 Test Suite and Structural Verification

Our testing framework consists of two driver programs in `src/`: `testbench_figure.c` for small-scale structural topology and edge-case verification corresponding to TC-1 and TC-2, and `testbench_time.c` for large-scale stress testing and runtime benchmarking across $N \in [1{,}000, 50{,}000]$ corresponding to TC-3 through TC-6. Table 2 summarizes the complete test suite.

Table 2: Summary of functional and performance test cases.

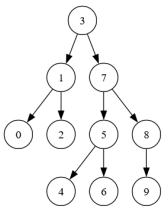
ID	Test Purpose	Expected Result	Actual Behavior	Status
TC-1	Topology visualization for $N = 10$ sequential insertions	BST forms a right chain; AVL, Splay, RB, and B$^+$ keep invariants	Generated DOT and PNG topologies match theoretical structures in Figure 3	Pass
TC-2	Duplicate and absent key edge cases	Duplicate insertions ignored; deleting absent key 20 leaves tree intact	Tree size and key order remain unchanged without memory leaks	Pass
TC-3	Monotone insertion stack safety at $N = 50{,}000$	No stack overflow on degenerate BST chain	Iterative BST solved recursive stack crash at $N \approx 26{,}000$	Corrected
TC-4	Workload 1: Increasing insert and increasing delete	BST insert $\Theta(N^2)$, delete $\Theta(N)$; balanced trees $O(N \log N)$	Measured curves and CSV timings match theoretical predictions	Pass
TC-5	Workload 2: Increasing insert and reverse delete	BST insert $\Theta(N^2)$, reverse delete $\Theta(N^2)$; Splay $\Theta(N)$	BST takes 1.78 s for reverse delete at $N = 50{,}000$; Splay takes 0.65 ms	Pass
TC-6	Workload 3: Random insert and random delete	All five structures run in $O(N \log N)$ expected time	All structures finish under 21 ms at $N = 50{,}000$; BST is fastest	Pass

Figure 3 shows the Graphviz structural diagrams generated by `testbench_figure.c` on small sequential inputs, using keys $0, \ldots, 9$ for BST, AVL, Splay, and B$^+$ Tree, and keys $10, \ldots, 19$ with key 15 deleted for the Red-Black Tree. One can visually confirm that the unbalanced BST degenerates into a right-leaning spine, the Splay Tree splays each sequentially inserted key to the root to form a left-leaning spine, the AVL and Red-Black Trees remain tightly balanced, and the order-4 B$^+$ Tree links all keys across two levels of leaf and internal routing blocks.

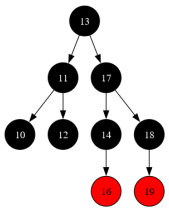


(a) BST

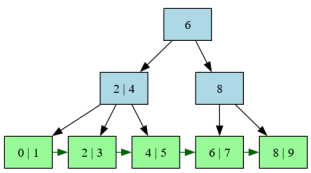
(b) Splay Tree



(c) AVL Tree



(d) Red-Black Tree



(e) Order-4 B$^+$ Tree with lightblue internal routing nodes and palegreen linked leaves

Figure 3: Tree topologies exported by `testbench_figure.c` and rendered via Graphviz.

3.2 Performance Benchmark Setup and Runtime Table

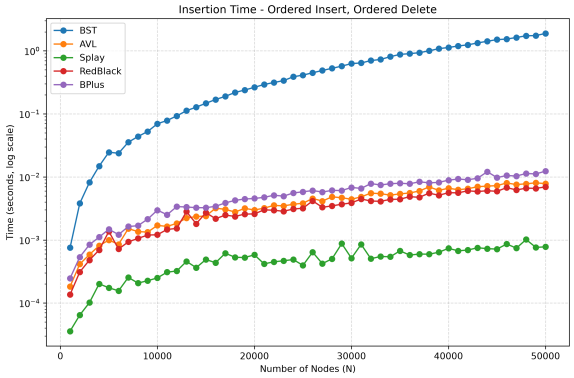
The benchmark was evaluated over input sizes $N \in \{1{,}000, 2{,}000, \ldots, 50{,}000\}$ with a step size of 1,000, with each data point averaged over 3 independent repetitions. For the random workload, identical pseudo-random permutations were generated via the Fisher–Yates shuffle to ensure that all five trees process the exact same key sequences. Table 3 reports the measured insertion and deletion times in milliseconds at representative input sizes ($N \in \{1{,}000, 10{,}000, 20{,}000, 30{,}000, 40{,}000, 50{,}000\}$) across all three scenarios.

Table 3: Execution time comparison in milliseconds across scenarios and input sizes N.

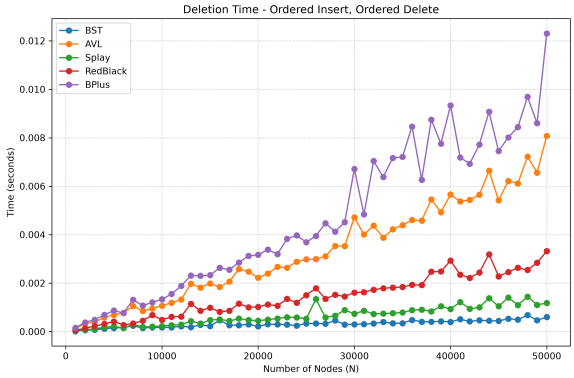
Scenario	Tree	$N = 1{,}000$	$N = 10{,}000$	$N = 20{,}000$	$N = 30{,}000$	$N = 40{,}000$	$N = 50{,}000$
Scenario 1: Ordered Insertion Time, ms	BST	0.754	69.678	264.054	628.587	1129.765	1888.738
	AVL	0.183	1.702	2.992	4.424	6.619	7.833
	Splay	0.036	0.250	0.581	0.514	0.736	0.778
	Red-Black	0.136	1.210	2.590	3.873	5.615	6.942
	B^+ Tree	0.247	2.972	4.557	6.809	8.891	12.355
Scenario 1: Ordered Deletion Time, ms	BST	0.016	0.171	0.217	0.301	0.399	0.599
	AVL	0.130	1.069	2.221	4.712	5.656	8.075
	Splay	0.044	0.227	0.449	0.735	0.929	1.177
	Red-Black	0.054	0.485	1.021	1.609	2.931	3.319
	B^+ Tree	0.159	1.332	3.167	6.714	9.338	12.306
Scenario 2: Reverse Deletion Time, ms	BST	0.684	63.698	251.397	590.257	1139.824	1777.648
	AVL	0.121	1.130	2.412	4.375	5.891	8.341
	Splay	0.018	0.110	0.227	0.336	0.391	0.646
	Red-Black	0.071	0.522	1.059	2.438	2.246	3.177
	B^+ Tree	0.157	1.281	3.182	5.975	6.615	8.095
Scenario 3: Random Insertion Time, ms	BST	0.090	0.843	1.941	3.838	4.015	5.524
	AVL	0.334	2.155	4.478	9.397	10.925	12.785
	Splay	0.166	1.858	3.941	7.184	8.619	10.446
	Red-Black	0.179	1.588	3.563	5.608	7.387	10.409
	B^+ Tree	0.276	3.000	7.625	12.549	16.264	19.595
Scenario 3: Random Deletion Time, ms	BST	0.074	0.764	1.753	2.985	4.417	5.359
	AVL	0.276	1.851	4.748	7.178	10.779	13.538
	Splay	0.198	1.371	3.342	6.337	7.656	10.079
	Red-Black	0.239	1.095	2.456	4.473	5.650	7.303
	B^+ Tree	0.235	2.548	6.142	11.828	16.430	20.871

3.3 Runtime Plots vs. Input Size

Figures 4 to 6 present the six runtime curves generated by `visualization.py` across $N \in [1{,}000, 50{,}000]$.



(a) Ordered insertion time (log scale)

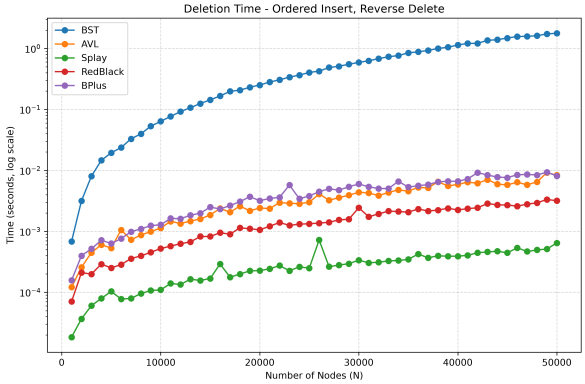


(b) Ordered deletion time

Figure 4: Scenario 1: Increasing-order insertion followed by increasing-order deletion.

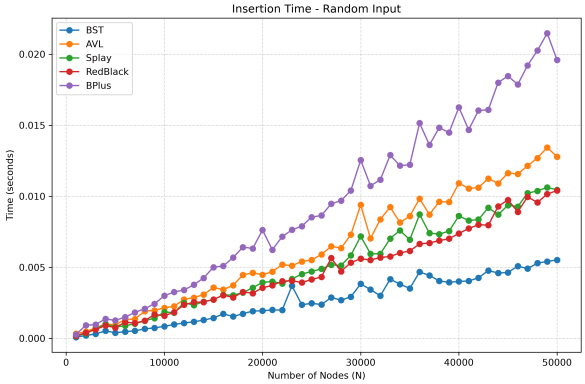


(a) Ordered insertion time (log scale)

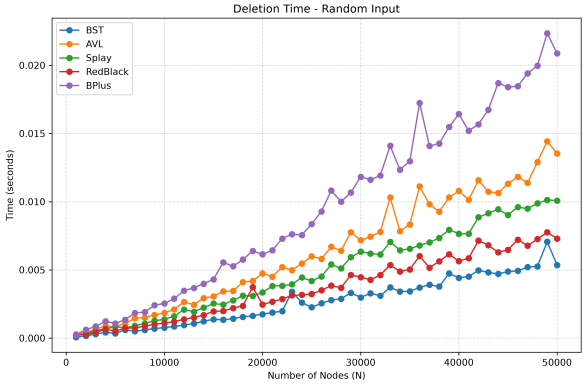


(b) Reverse deletion time (log scale)

Figure 5: Scenario 2: Increasing-order insertion followed by reverse-order deletion.



(a) Random insertion time



(b) Random deletion time

Figure 6: Scenario 3: Uniform random-order insertion followed by random-order deletion.

Chapter 4. Analysis and Comments

4.1 Time and Space Complexity Analysis

Table 4 summarizes the theoretical time and space complexities of a single dictionary operation and a sequence of N operations for all five structures.

Table 4: Theoretical asymptotic time and space complexities.

Structure	Average Op	Worst-Case Op	Sorted Ins + Same Del	Sorted Ins + Rev Del
Unbalanced BST	$\Theta(\log N)$	$\Theta(N)$	$\Theta(N^2)$ ins / $\Theta(N)$ del	$\Theta(N^2)$ ins / $\Theta(N^2)$ del
AVL Tree	$\Theta(\log N)$	$\Theta(\log N)$	$\Theta(N \log N)$	$\Theta(N \log N)$
Splay Tree	$\Theta(\log N)$ amort.	$\Theta(N)$ single	$\Theta(N)$ ins / $\Theta(N)$ del	$\Theta(N)$ ins / $\Theta(N)$ del
Red-Black Tree	$\Theta(\log N)$	$\Theta(\log N)$	$\Theta(N \log N)$	$\Theta(N \log N)$
Order-4 B^+ Tree	$\Theta(\log N)$	$\Theta(\log N)$	$\Theta(N \log N)$	$\Theta(N \log N)$

Space Complexity. Every dynamic tree stores $O(N)$ nodes in heap memory. Specifically:

- **BST and Splay Tree** require the minimal node size of 24 bytes on 64-bit targets, comprising one integer key and two child pointers, with $O(1)$ auxiliary stack space for our iterative BST and $O(H)$ recursion stack space for Splay.
- **AVL Tree** adds two integer fields `height` and `BF`, taking 32 bytes per node and $O(\log N)$ recursion stack space.
- **Red-Black Tree** adds a `color` field and a `parent` pointer for 40 bytes per node, while its fix-up loops are purely iterative and use $O(1)$ auxiliary space after node insertion.
- **B^+ Tree** allocates dynamic key and child pointer arrays `keys[4]` and `children[5]` per node, consuming more heap allocations per key for small order $M = 4$, though with reduced tree depth $\log_{2\ldots4} N$.

4.2 Workload-Specific Empirical Behavior

4.2.1 Asymmetry of Unbalanced BST Under Ordered vs. Reverse Deletion

One of the most striking phenomena in Figures 4 and 5 is the dramatic difference in Unbalanced BST deletion time between Scenario 1 and Scenario 2:

Key Finding 1: Why BST Ordered Deletion Takes $\Theta(N)$ While Reverse Deletion Takes $\Theta(N^2)$

- **Ordered Insertion, $\Theta(N^2)$ total:** Inserting $0, 1, \ldots, N-1$ appends each new key to the rightmost leaf of a right-skewed chain. The k-th insertion traverses k nodes, yielding $\sum_{k=0}^{N-1} k = \frac{N(N-1)}{2} = \Theta(N^2)$ comparisons. Empirically, when N increases 5× from 10,000 to 50,000, BST insertion time increases by $1888.74/69.68 \approx 27.1\times$, which closely matches the theoretical factor $5^2 = 25$ together with cache-miss amplification.
- **Scenario 1 — Increasing Deletion, $\Theta(N)$ total:** In a right-skewed chain $0 \to 1 \to \cdots \to N-1$, the smallest element is always the **current root**. Deleting keys in increasing order $0, 1, \ldots, N-1$ repeatedly removes the root node in $O(1)$ time by advancing `root` $\leftarrow$ `root->Right` without traversing the chain. Thus N deletions take $\Theta(N)$ total time, reaching only 0.60 ms at $N = 50{,}000$ and making BST the fastest structure in Figure 4b.
- **Scenario 2 — Reverse Deletion, $\Theta(N^2)$ total:** Deleting keys in decreasing order $N-1, N-2, \ldots, 0$ always targets the **deepest leaf** at the bottom of the right chain. The k-th deletion must traverse $N-k$ nodes from the root, resulting in $\sum_{k=1}^{N}(N-k) = \Theta(N^2)$ total

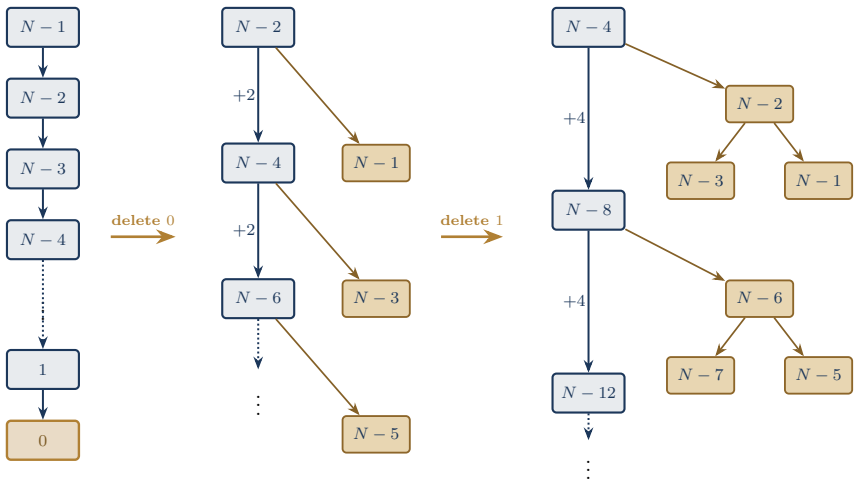
time, or 1.78 s at $N = 50{,}000$ in Figure 5b.

4.2.2 Sequential Locality Advantage of Splay Trees

Across both Scenario 1 and Scenario 2, the **Splay Tree** is by far the fastest structure for sorted insertions and reverse deletions, taking only 0.78 ms and 0.65 ms at $N = 50{,}000$, which is nearly an order of magnitude faster than Red-Black and AVL Trees.

Key Finding 2: $\Theta(N)$ Sequential Access Performance of Splay Trees

- **Increasing Insertion, $\Theta(N)$ total:** Because `insertSplay` splays each newly inserted key k to the root via split, when $k+1$ arrives, the root already holds the largest existing key k and has an empty right subtree. Thus `findSplay(root, k+1)` immediately terminates in $O(1)$ time at the root, and `insertSplay` attaches $k+1$ as the new root with k as its left child in $O(1)$ time. This constructs a left-skewed spine in $\Theta(N)$ total time, as shown in Figure 3b.
- **Reverse Deletion, $\Theta(N)$ total:** At the end of increasing insertion, the tree is a left chain rooted at $N-1$ with parent-to-left-child order $N-1 \to N-2 \to \cdots \to 0$. Deleting in reverse order $N-1, N-2, \ldots, 0$ always deletes the current root in $O(1)$ time.
- **Increasing Deletion, $\Theta(N)$ amortized:** Even when deleting the deepest leaf on the left spine, the first `findSplay` performs a cascade of *zig-zig* rotations that folds the path and roughly **halves the tree height**; the next deepest deletion halves it again, so only a few such steps leave a shallow tree and the remaining deletions cost $O(1)$ amortized each [2], finishing in 1.18 ms at $N = 50{,}000$. Figure 7 draws the chain in terms of N: after deleting 0 the left spine steps by +2, and after deleting 1 it steps by +4, so the height falls from $N-1$ to about $N/2$ and then about $N/4$.



(a) Insert $0, \ldots, N-1$ (b) After delete 0 (c) After delete 1

Figure 7: Zig-zig folding of the left spine: deleting 0 and then 1 rebuilds the left path in steps of +2 and then +4.

4.2.3 Trade-offs Among Balanced Trees under Random Workloads

In Scenario 3, shown in Figure 6, random permutations eliminate adversarial skew, yielding the runtime ranking **BST < Red-Black < Splay < AVL < B$^+$ Tree** at $N = 50{,}000$:

1. **Unbalanced BST:** A random BST has expected depth $2 \ln N \approx 1.39 \log_2 N$ and performs zero

rotations or metadata updates, making it the fastest structure when input randomness is guaranteed.

2. **Red-Black Tree vs. AVL Tree:** At $N = 50{,}000$, the Red-Black Tree takes 10.41 ms for insertion and 7.30 ms for deletion, whereas the AVL Tree takes 12.79 ms and 13.54 ms. Both maintain $O(\log N)$ height, but Red-Black Trees consistently outperform AVL Trees by 20%–45% on both ordered and random updates. While AVL deletion may trigger $\Theta(\log N)$ rotations propagating to the root, Red-Black insertion requires at most 2 rotations and Red-Black deletion requires at most 3 rotations, resolving most imbalances via fast color flips.
3. **Splay Tree:** Under uniform random access, splaying loses its locality advantage and pays constant-factor rotation overhead on every lookup, performing comparably to Red-Black on insertion and slightly slower on deletion.
4. **B$^+$ Tree:** The order-4 B$^+$ Tree is the slowest in our benchmarks. Each split, borrow, or merge allocates dynamic arrays, shifts elements linearly inside `keys[]` and `children[]`, and invokes `refreshBPlusAncestors` to walk down `children[0]` and update separator keys, incurring much higher constant-factor overhead per operation than binary tree pointer rotations.

4.3 Further Possible Improvements

Based on our implementation and profiling results, several engineering improvements can be pursued:

1. **Eliminating Redundant Separator Updates in B$^+$ Tree:** Currently, `refreshBPlusAncestors` invokes `firstBPlusKey` on every child of every ancestor after each update. Caching the minimum key in each node would reduce per-update overhead by a large constant factor.
2. **Using Fixed-Size Arrays in B$^+$ Tree Nodes:** Replacing per-node `malloc` calls for `keys` and `children` with fixed-size arrays `int keys[M]` and `BPlusTreeNode* children[M+1]` inside the node struct would avoid repeated heap allocations on every node creation.
3. **Top-Down Splaying and Memory Pool Allocation:** Replacing recursive splaying with iterative top-down splaying would remove recursion stack overhead, and using a pre-allocated node memory pool (**from OOP Lab 7 last semester**) across all trees would isolate tree algorithm cost from operating-system `malloc` and `free` overhead.

Author List and Declaration

Author List

- **Yixiang Chen:** Implemented the five search tree header libraries, the timing and topology test-benches, and the Python plotting script.
- **Runhui Lin:** Conducted experimental verification, analyzed complexity and empirical runtime trends across workloads, and authored the project report.

Declaration

We hereby declare that all the work done in this project titled "Performance Comparison and Analysis of Search Trees" is of our independent effort as a group.

Signatures

Signature of Group Member 1

Signature of Group Member 2

References

[1] Thomas H. Cormen et al. *Introduction to Algorithms*. 4th ed. MIT Press, 2022.

[2] Course Teaching Team of Advanced Data Structures and Algorithm Analysis. *Course Lecture Slides: AVL Trees, Splay Trees, Red-Black Trees, and B+ Trees*. College of Computer Science and Technology, Zhejiang University. 2026.

[3] Mark Allen Weiss. *Data Structures and Algorithm Analysis in C*. 2nd ed. Addison-Wesley, 1996.

Appendix A. Source Code

All complete source files are located in `project/project1/src/`. Below we include the core C implementations of the five search trees and the performance benchmark driver (excluding boilerplate Graphviz DOT export helpers for brevity).

A.1 Unbalanced Binary Search Tree (`binaray_search_tree.h`)

```
#endif
}
#endif

/*
 * Node structure for an unbalanced Binary Search Tree.
 * Stores an integer key and pointers to the left and right subtrees.
 */
typedef struct BSTreeNode BSTreeNode;
struct BSTreeNode{
    int val; /* Integer key stored in the node */
    BSTreeNode* Left; /* Pointer to left child (keys < val) */
    BSTreeNode* Right; /* Pointer to right child (keys > val) */
};

/*
 * Recursive implementations kept for reference.
 * WARNING: An ordered input makes an ordinary BST degenerate into a chain.
 * These recursive functions may then create O(N) call frames and cause a
 * stack overflow when N is large (around 26000 with the default call stack).
 *
BSTreeNode* insertBST(BSTreeNode* root, int val) { // Insert a value into the binary search tree
    if (root == NULL) {
        root = (BSTreeNode*)malloc(sizeof(BSTreeNode)); // Allocate memory for the new node
        root->val = val;
        root->Left = NULL;
        root->Right = NULL;
        return root;
    }
    if (val < root->val) { // If the value is smaller, insert into the left subtree
        root->Left = insertBST(root->Left, val);
    } else if (val > root->val) { // If the value is larger, insert into the right subtree
        root->Right = insertBST(root->Right, val);
    } else {
        // Duplicate value, do nothing (BST does not allow duplicates)
    }
    return root;
}

BSTreeNode* deleteBST(BSTreeNode* root, int val) { // Delete a value from the binary search tree
    if (root == NULL) {
        return root; // Value not found
    }
    if (val < root->val) { // If the value is smaller, search in the left subtree
        root->Left = deleteBST(root->Left, val);
    } else if (val > root->val) { // If the value is larger, search in the right subtree
        root->Right = deleteBST(root->Right, val);
    } else { // Found the node to delete
        if (root->Left == NULL) { // Node has no left child
            BSTreeNode* temp = root->Right;
            free(root);
            root = temp;
            return root;
        } else if (root->Right == NULL) { // Node has no right child
            BSTreeNode* temp = root->Left;
            free(root);
            root = temp;
            return root;
        } else { // Node has two children, find the inorder successor (smallest node in the right subtree)
            BSTreeNode* temp = root->Right;
            while (temp->Left != NULL) {
                temp = temp->Left; // find the smallest node in the right subtree
            }
```

```
            root->val = temp->val;
            root->Right = deleteBST(root->Right, temp->val); // Delete the inorder successor from the right subtree
            return root;
        }
    }
    return root; // Return the updated root after deletion
}
*/

/*
 * Insert a key into the BST iteratively.
 * Uses O(1) auxiliary stack space to prevent stack overflow on skewed inputs.
 */
BSTreeNode* insertBST(BSTreeNode* root, int val) {
    BSTreeNode* parent = NULL;
    BSTreeNode* current = root;

    /* Walk down the tree from the root to locate the insertion point */
    while (current != NULL) {
        parent = current;
        if (val < current->val) {
            current = current->Left; /* Move to left child */
        } else if (val > current->val) {
            current = current->Right; /* Move to right child */
        } else {
            return root; /* Duplicate key found; ignore insertion */
        }
    }

    /* Allocate and initialize the new leaf node */
    BSTreeNode* newNode = (BSTreeNode*)malloc(sizeof(BSTreeNode));
    if (newNode == NULL) return root; /* Memory allocation failed */
    newNode->val = val;
    newNode->Left = NULL;
    newNode->Right = NULL;

    /* Attach the new node to its parent (or set as root if tree was empty) */
    if (parent == NULL) {
        return newNode; /* The original tree was empty */
    } else if (val < parent->val) {
        parent->Left = newNode;
    } else {
        parent->Right = newNode;
    }
    return root;
}

/*
 * Delete a key from the BST iteratively.
 * Handles leaf nodes, one-child nodes, and two-child nodes (via in-order successor).
 */
BSTreeNode* deleteBST(BSTreeNode* root, int val) {
    BSTreeNode* parent = NULL;
    BSTreeNode* current = root;

    /* Search for the target node while keeping track of its parent */
    while (current != NULL && current->val != val) {
        parent = current;
        if (val < current->val) {
            current = current->Left;
        } else {
            current = current->Right;
        }
    }
    if (current == NULL) return root; /* Target key not found in the tree */

    /* If the target node has two children, reduce to deleting its in-order successor */
    if (current->Left != NULL && current->Right != NULL) {
        /* Find the minimum node in the right subtree (which has no left child) */
        BSTreeNode* successorParent = current;
        BSTreeNode* successor = current->Right;
        while (successor->Left != NULL) {
            successorParent = successor;
            successor = successor->Left;
        }
```

```
        /* Copy the successor's key into the current node and splice out the successor */
        current->val = successor->val;
        parent = successorParent;
        current = successor;
    }

    /* Connect the single child (or NULL) directly to the removed node's parent */
    BSTreeNode* child = current->Left != NULL ? current->Left : current->Right;
    if (parent == NULL) {
        root = child; /* Deleting the root node */
```

A.2 AVL Tree (`AVL_tree.h`)

```
    char command[1200];
    snprintf(command, sizeof(command), "\"%s\" -Tpng \"%s\" -o \"%s\"", exe, dotPath, pngPath);
    return system(command) == 0 ? 0 : -1;
#endif
}
#endif

/*
 * AVL Tree node definition.
 * Maintains subtree height and balance factor BF = height(Left) - height(Right)
 * so that |BF| <= 1 holds at every node after rebalancing.
 */
typedef struct AVLTreeNode AVLTreeNode;
struct AVLTreeNode{
    int val; /* Integer key stored in the node */
    int height; /* Height of the subtree rooted at this node (leaf = 0) */
    int BF; /* Balance factor: height(Left) - height(Right) */
    AVLTreeNode* Left; /* Pointer to the left subtree */
    AVLTreeNode* Right; /* Pointer to the right subtree */
};

/*
 * Return the height of a node, or -1 for an empty (NULL) subtree.
 */
int CalculateHeight(AVLTreeNode* node) {
    return node == NULL ? -1 : node->height;
}

/*
 * Compute the balance factor BF(node) = height(Left) - height(Right).
 */
int CalculateBF(AVLTreeNode* node) {
    if (node == NULL) {
        return 0;
    }
    return CalculateHeight(node->Left) - CalculateHeight(node->Right);
}

/*
 * LL Single Right Rotation:
 * Restores balance when the left-left subtree becomes too heavy (BF = +2, left BF >= 0).
 * Lifts the left child L as the new subtree root and updates heights and balance factors.
 */
AVLTreeNode* singleRotationLL(AVLTreeNode* root){
    AVLTreeNode* L = root->Left;
    root->Left = L->Right; /* Move L's right subtree to root's left */
    L->Right = root; /* Place old root as L's right child */

    /* Recompute height and balance factor of the demoted root first */
    root->height = (CalculateHeight(root->Left) > CalculateHeight(root->Right)
                    ? CalculateHeight(root->Left) : CalculateHeight(root->Right)) + 1;
    root->BF = CalculateBF(root);

    /* Recompute height and balance factor of the new subtree root L */
    L->height = (CalculateHeight(L->Left) > CalculateHeight(L->Right)
                 ? CalculateHeight(L->Left) : CalculateHeight(L->Right)) + 1;
    L->BF = CalculateBF(L);
    return L;
}

```

```
/*
 * RR Single Left Rotation:
 * Restores balance when the right-right subtree becomes too heavy (BF = -2, right BF <= 0).
 * Lifts the right child R as the new subtree root and updates heights and balance factors.
 */
AVLTreeNode* singleRotationRR(AVLTreeNode* root){
    AVLTreeNode* R = root->Right;
    root->Right = R->Left; /* Move R's left subtree to root's right */
    R->Left = root; /* Place old root as R's left child */

    /* Recompute height and balance factor of the demoted root first */
    root->height = (CalculateHeight(root->Left) > CalculateHeight(root->Right)
                 ? CalculateHeight(root->Left) : CalculateHeight(root->Right)) + 1;
    root->BF = CalculateBF(root);

    /* Recompute height and balance factor of the new subtree root R */
    R->height = (CalculateHeight(R->Left) > CalculateHeight(R->Right)
              ? CalculateHeight(R->Left) : CalculateHeight(R->Right)) + 1;
    R->BF = CalculateBF(R);
    return R;
}

/*
 * LR Double Rotation (Left-Right):
 * Left-rotates the left child to convert LR imbalance into LL, then right-rotates root.
 */
AVLTreeNode* doubleRotationLR(AVLTreeNode* root){
    root->Left = singleRotationRR(root->Left);
    return singleRotationLL(root);
}

/*
 * RL Double Rotation (Right-Left):
 * Right-rotates the right child to convert RL imbalance into RR, then left-rotates root.
 */
AVLTreeNode* doubleRotationRL(AVLTreeNode* root){
    root->Right = singleRotationLL(root->Right);
    return singleRotationRR(root);
}

/*
 * Insert a key into the AVL tree and rebalance along the return path.
 */
AVLTreeNode* insertAVL(AVLTreeNode* root, int val){
    /* Base case: allocate a new leaf node with height 0 and BF 0 */
    if (root == NULL) {
        AVLTreeNode* newNode = (AVLTreeNode*)malloc(sizeof(AVLTreeNode));
        newNode->val = val;
        newNode->height = 0;
        newNode->BF = 0;
        newNode->Left = NULL;
        newNode->Right = NULL;
        return newNode;
    }

    /* Recursively descend into the left or right subtree */
    if (val < root->val) {
        root->Left = insertAVL(root->Left, val);
    } else if (val > root->val) {
        root->Right = insertAVL(root->Right, val);
    } else {
        /* Duplicate key already exists; no insertion needed */
        return root;
    }

    /* Update height and balance factor of the current ancestor */
    root->height = (CalculateHeight(root->Left) > CalculateHeight(root->Right)
                 ? CalculateHeight(root->Left) : CalculateHeight(root->Right)) + 1;
    root->BF = CalculateBF(root);

    /* Left-heavy imbalance (BF == +2): apply LR or LL rotation */
    if (root->BF > 1) {
        if (CalculateBF(root->Left) < 0) {
            return doubleRotationLR(root); /* Left-Right case */
        } else {
```

```
            return singleRotationLL(root); /* Left-Left case */
        }
    }

    /* Right-heavy imbalance (BF == -2): apply RL or RR rotation */
    if (root->BF < -1) {
        if (CalculateBF(root->Right) > 0) {
            return doubleRotationRL(root); /* Right-Left case */
        } else {
            return singleRotationRR(root); /* Right-Right case */
        }
    }

    return root;
}

/*
 * Delete a key from the AVL tree and rebalance all affected ancestors.
 */
AVLTreeNode* deleteAVL(AVLTreeNode* root, int val){
    if (root == NULL){
        return NULL; /* Key not found */
    }

    /* Search for the node to delete in the appropriate subtree */
    if (val < root->val) {
        root->Left = deleteAVL(root->Left, val);
    } else if (val > root->val) {
        root->Right = deleteAVL(root->Right, val);
    } else {
        /* Target node found */
        if (root->Left == NULL) {
            /* Only right child or leaf: replace node with its right child */
            AVLTreeNode* temp = root->Right;
            free(root);
            return temp;
        } else if (root->Right == NULL) {
            /* Only left child: replace node with its left child */
            AVLTreeNode* temp = root->Left;
            free(root);
            return temp;
        } else {
            /* Two children: replace value with in-order successor (minimum of right subtree) */
            AVLTreeNode* temp = root->Right;
            while (temp->Left != NULL) {
                temp = temp->Left;
            }
            root->val = temp->val;
            root->Right = deleteAVL(root->Right, temp->val);
        }
    }

    /* Update height and balance factor after deletion in subtree */
    root->height = (CalculateHeight(root->Left) > CalculateHeight(root->Right)
                ? CalculateHeight(root->Left) : CalculateHeight(root->Right)) + 1;
    root->BF = CalculateBF(root);

    /* Restore AVL balance if the current node became left-heavy (BF == +2) */
    if (root->BF > 1) {
        if (CalculateBF(root->Left) < 0) {
            return doubleRotationLR(root);
        } else {
            return singleRotationLL(root);
        }
    }

    /* Restore AVL balance if the current node became right-heavy (BF == -2) */
```

A.3 Splay Tree (`splay_tree.h`)

```
    const char* const argv[] = {exe, "-Tpng", dotPath, "-o", pngPath, NULL};
    return _spawnvp(_P_WAIT, exe, argv) == 0 ? 0 : -1;
#else
```

```
    char command[1200]; snprintf(command, sizeof(command), "\"%s\" -Tpng \"%s\" -o \"%s\"", exe, dotPath, pngPath); return
        system(command) == 0 ? 0 : -1;
#endif
}
#endif

/*
 * Splay Tree node structure.
 * Requires no explicit height or color metadata; amortized O(log N) balance
 * is maintained dynamically by splaying accessed nodes to the root.
 */
typedef struct SplayTreeNode SplayTreeNode;

struct SplayTreeNode{
    int key; /* Integer key stored in the node */
    SplayTreeNode* left; /* Pointer to the left subtree */
    SplayTreeNode* right; /* Pointer to the right subtree */
};

/*
 * Single right rotation (zig on left child): lifts root->left to root.
 */
SplayTreeNode* singleRotateLL(SplayTreeNode* root){
    SplayTreeNode* newRoot = root->left;
    root->left = newRoot->right;
    newRoot->right = root;
    return newRoot;
}

/*
 * Single left rotation (zag on right child): lifts root->right to root.
 */
SplayTreeNode* singleRotateRR(SplayTreeNode* root){
    SplayTreeNode* newRoot = root->right;
    root->right = newRoot->left;
    newRoot->left = root;
    return newRoot;
}

/*
 * Search for 'key' and splay the matching node (or last visited node) to the root.
 * Applies two-level zig-zig and zig-zag rotations along the recursion path.
 */
SplayTreeNode* findSplay(SplayTreeNode* root, int key){
    if(!root || root->key == key) return root;

    /* Target key lies in the left subtree */
    if(key < root->key){
        if(root->left == NULL) return root; /* Key is not in the tree; splay last visited */

        if(key < root->left->key){
            /* Zig-Zig (Left-Left): splay in left-left grandchild, then rotate grandparent first */
            root->left->left = findSplay(root->left->left, key);
            root = singleRotateLL(root);
        } else if(key > root->left->key){
            /* Zig-Zag (Left-Right): splay in left-right grandchild, then rotate parent first */
            root->left->right = findSplay(root->left->right, key);
            if(root->left->right != NULL)
                root->left = singleRotateRR(root->left);
        }

        /* Perform the second rotation (or single Zig at root) */
        return root->left == NULL ? root : singleRotateLL(root);
    } else {
        /* Target key lies in the right subtree */
        if(root->right == NULL) return root; /* Key is not in the tree; splay last visited */

        if(key > root->right->key){
            /* Zag-Zag (Right-Right): splay in right-right grandchild, then rotate grandparent first */
            root->right->right = findSplay(root->right->right, key);
            root = singleRotateRR(root);
        } else if(key < root->right->key){
            /* Zag-Zig (Right-Left): splay in right-left grandchild, then rotate parent first */
            root->right->left = findSplay(root->right->left, key);
            if(root->right->left != NULL)
```

```
            root->right = singleRotateLL(root->right);
        }

        /* Perform the second rotation (or single Zag at root) */
        return root->right == NULL ? root : singleRotateRR(root);
    }
}

/*
 * Insert 'key' into the Splay Tree via splay-and-split.
 * Splays the closest existing key to the root, then splits the tree around the new root.
 */
SplayTreeNode* insertSplay(SplayTreeNode* root, int key){
    /* If the tree is empty, create and return a single-node tree */
    if(!root){
        root = (SplayTreeNode*)malloc(sizeof(SplayTreeNode));
        if(root == NULL) return NULL;
        root->key = key;
        root->left = NULL;
        root->right = NULL;
        return root;
    }

    /* Splay the key (or the closest neighbor) to the root */
    root = findSplay(root, key);
    if(root->key == key) return root; /* Key already exists; ignore duplicate */

    /* Allocate the new root node and split the old root into left and right subtrees */
    SplayTreeNode* newRoot = (SplayTreeNode*)malloc(sizeof(SplayTreeNode));
    if(newRoot == NULL) return root;
    newRoot->key = key;

    if(key < root->key){
        /* New key is smaller than splayed root: old root becomes right child */
        newRoot->left = root->left;
        newRoot->right = root;
        root->left = NULL;
    } else {
        /* New key is larger than splayed root: old root becomes left child */
        newRoot->right = root->right;
        newRoot->left = root;
        root->right = NULL;
    }
    return newRoot;
}

/*
 * Delete 'key' from the Splay Tree via splay-and-join.
 * Splays target key to the root, removes it, and joins its left and right subtrees.
 */
SplayTreeNode* deleteSplay(SplayTreeNode* root, int key){
    if(!root) return NULL;

    /* Splay the target key to the root; if absent, return the splayed tree */
    root = findSplay(root, key);
    if(root->key != key) return root;

    SplayTreeNode* newRoot;
    if(root->left == NULL){
        /* No left subtree: promote right subtree directly */
        newRoot = root->right;
    } else {
        /* Splay the maximum element of the left subtree (all keys < key) to its root */
        newRoot = findSplay(root->left, key);
        /* Attach the original right subtree to the new root's empty right link */
        newRoot->right = root->right;
    }
    free(root);
```

A.4 Red-Black Tree (`red_black_tree.h`)

```
    const char* exe = getenv("GRAPHVIZ_DOT"); if (exe == NULL || exe[0] == '\0') exe = "dot";
#ifdef _WIN32
```

```
    const char* const argv[] = {exe, "-Tpng", dotPath, "-o", pngPath, NULL};
    return _spawnvp(_P_WAIT, exe, argv) == 0 ? 0 : -1;
#else
    char command[1200]; snprintf(command, sizeof(command), "\"%s\" -Tpng \"%s\" -o \"%s\"", exe, dotPath, pngPath); return
        system(command) == 0 ? 0 : -1;
#endif
}
#endif

/* Node color constants */
#define BLACK 0
#define RED 1
typedef struct RedBlackTreeNode RedBlackTreeNode;

/*
```

```
    fprintf(fp, "digraph RED_BLACK {\n");
    fprintf(fp, " graph [rankdir=TB];\n");
    fprintf(fp, " node [shape=circle];\n");

    exportRBDOT(root, fp);

    fprintf(fp, "}\n");
    fclose(fp);

    int ret = treeRenderGraphviz(dotPath, pngPath);

    if (ret == 0) {
        printf("Red-black tree visualization saved to %s\n", pngPath);
    } else {
        fprintf(stderr,
                "Graphviz rendering failed. Make sure Graphviz is installed "
                "and the 'dot' command is available in PATH.\n"
                "The DOT source was still saved to %s\n",
                dotPath);
    }
}

/*
 * Single right rotation (LL rotation) at 'root', updating parent pointers.
 */
RedBlackTreeNode* singlerotationLL(RedBlackTreeNode* root){
    RedBlackTreeNode* oldParent = root->parent;
    RedBlackTreeNode* newRoot = root->left;
    root->left = newRoot->right;
    if(root->left)
        root->left->parent = root;
    newRoot->right = root;
    newRoot->parent = oldParent;
    root->parent = newRoot;
    /* Reconnect the rotated subtree to oldParent */
    if (oldParent != NULL) {
        if (oldParent->left == root) oldParent->left = newRoot;
        else if (oldParent->right == root) oldParent->right = newRoot;
    }
    return newRoot;
}

/*
 * Single left rotation (RR rotation) at 'root', updating parent pointers.
 */
RedBlackTreeNode* singlerotationRR(RedBlackTreeNode* root){
    RedBlackTreeNode* oldParent = root->parent;
    RedBlackTreeNode* newRoot = root->right;
    root->right = newRoot->left;
    if(root->right)
        root->right->parent = root;
    newRoot->left = root;
    newRoot->parent = oldParent;
    root->parent = newRoot;
    /* Reconnect the rotated subtree to oldParent */
    if (oldParent != NULL) {
        if (oldParent->left == root) oldParent->left = newRoot;
        else if (oldParent->right == root) oldParent->right = newRoot;
    }
```

```
    return newRoot;
}

/*
 * Standard BST insertion helper that colors the new leaf RED and outputs its pointer.
 */
static RedBlackTreeNode* insertRBNode(RedBlackTreeNode* root, int key,
                                      RedBlackTreeNode** inserted) {
    if (root == NULL) {
        RedBlackTreeNode* node = (RedBlackTreeNode*)malloc(sizeof(*node));
        if (node == NULL) return NULL;
        node->key = key;
        node->color = RED; /* New leaves are always colored RED initially */
        node->left = node->right = node->parent = NULL;
        *inserted = node;
        return node;
    }

    if (key < root->key) {
        root->left = insertRBNode(root->left, key, inserted);
        if (root->left != NULL) root->left->parent = root;
    } else if (key > root->key) {
        root->right = insertRBNode(root->right, key, inserted);
        if (root->right != NULL) root->right->parent = root;
    }
    return root;
}

/*
 * Climb parent pointers from 'node' to locate the current root of the tree.
 */
static RedBlackTreeNode* rbRoot(RedBlackTreeNode* node) {
    if (node == NULL || node->parent == NULL) return node;
    return rbRoot(node->parent);
}

/*
 * Restore Red-Black properties after inserting a RED leaf 'node'.
 * Resolves consecutive red-red violations (color(node) == RED && color(parent) == RED)
 * using at most two rotations.
 */
static RedBlackTreeNode* fixRedBlackAfterInsert(RedBlackTreeNode* node) {
    while (node != NULL && node->parent != NULL && node->parent->color == RED) {
        RedBlackTreeNode* parent = node->parent;
        RedBlackTreeNode* grandparent = parent->parent;
        if (grandparent == NULL) break;

        if (parent == grandparent->left) {
            /* Parent is the left child of grandparent */
            RedBlackTreeNode* uncle = grandparent->right;
            if (uncle != NULL && uncle->color == RED) {
                /* Case 1: Uncle is RED -> recolor parent, uncle, and grandparent */
                parent->color = BLACK;
                uncle->color = BLACK;
                grandparent->color = RED;
                node = grandparent;
            } else {
                if (node == parent->right) {
                    /* Case 2: Uncle is BLACK and node is inner (LR) child -> rotate parent */
                    singlerotationRR(parent);
                    parent = node;
                }
                /* Case 3: Uncle is BLACK and node is outer (LL) child -> recolor and rotate grandparent */
                parent->color = BLACK;
                grandparent->color = RED;
                singlerotationLL(grandparent);
                break;
            }
        } else {
            /* Symmetric cases when parent is the right child of grandparent */
            RedBlackTreeNode* uncle = grandparent->left;
            if (uncle != NULL && uncle->color == RED) {
                /* Case 1: Uncle is RED -> recolor parent, uncle, and grandparent */
                parent->color = BLACK;
                uncle->color = BLACK;
```

```
            grandparent->color = RED;
            node = grandparent;
        } else {
            if (node == parent->left) {
                /* Case 2: Uncle is BLACK and node is inner (RL) child -> rotate parent */
                singlerotationLL(parent);
                parent = node;
            }
            /* Case 3: Uncle is BLACK and node is outer (RR) child -> recolor and rotate grandparent */
            parent->color = BLACK;
            grandparent->color = RED;
            singlerotationRR(grandparent);
            break;
        }
    }
}

/* Ensure the tree root is always colored BLACK */
RedBlackTreeNode* root = rbRoot(node);
if (root != NULL) root->color = BLACK;
return root;
}

/*
 * Public API: Insert 'key' into the Red-Black Tree and return the updated root.
 */
RedBlackTreeNode* insertRedBlack(RedBlackTreeNode* root, int key) {
    RedBlackTreeNode* inserted = NULL;
    RedBlackTreeNode* treeRoot = insertRBNode(root, key, &inserted);
    if (treeRoot == NULL || inserted == NULL) return treeRoot;
    return fixRedBlackAfterInsert(inserted);
}

/*
 * Restore Red-Black properties after deleting a BLACK node.
 * The parameter 'node' carries the extra "double-black" token upward until resolved
 * via recoloring or at most three rotations across Cases 1-4.
 */
RedBlackTreeNode* fixRedBlackAfterDelete(RedBlackTreeNode* node) {
    if(node == NULL) return NULL;

    while(node->parent != NULL && node->color == BLACK) {
        RedBlackTreeNode* parent = node->parent;
        RedBlackTreeNode* sibling = (node == parent->left) ? parent->right : parent->left;

        /* Case 1: Sibling is RED -> recolor and rotate parent to obtain a BLACK sibling */
        if(sibling != NULL && sibling->color == RED) {
            sibling->color = BLACK;
            parent->color = RED;
            if(node == parent->left) {
                singlerotationRR(parent);
            } else {
                singlerotationLL(parent);
            }
            sibling = (node == parent->left) ? parent->right : parent->left;
        }

        if(sibling == NULL) {
            node = parent;
            continue;
        }

        /* Case 2: Sibling is BLACK and both nephews are BLACK -> recolor sibling RED */
        if((sibling->left == NULL || sibling->left->color == BLACK) &&
           (sibling->right == NULL || sibling->right->color == BLACK)) {
            sibling->color = RED;
            /* If parent was RED, absorb the extra black token at parent and terminate */
            if(parent->color == RED) {
                parent->color = BLACK;
                break;
            } else {
                /* Otherwise propagate the double-black token up to parent */
                node = parent;
                continue;
            }
```

```
        } else if(node == parent->left){
            /* Node is left child: check Case 3 (inner nephew RED, outer nephew BLACK) */
            if(sibling->right == NULL || sibling->right->color == BLACK){
                sibling->color = RED;
                if(sibling->left != NULL) sibling->left->color = BLACK;
                singlerotationLL(sibling);
                sibling = parent->right;
            }
            /* Case 4: Sibling is BLACK and outer (right) nephew is RED -> rotate parent and finish */
            sibling->color = parent->color;
            parent->color = BLACK;
            if(sibling->right != NULL) sibling->right->color = BLACK;
            singlerotationRR(parent);
            break;
        } else {
            /* Symmetric: Node is right child; check Case 3 (inner nephew RED, outer nephew BLACK) */
            if(sibling->left == NULL || sibling->left->color == BLACK){
                sibling->color = RED;
                if(sibling->right != NULL) sibling->right->color = BLACK;
                singlerotationRR(sibling);
                sibling = parent->left;
            }
            /* Case 4: Sibling is BLACK and outer (left) nephew is RED -> rotate parent and finish */
            sibling->color = parent->color;
            parent->color = BLACK;
            if(sibling->left != NULL) sibling->left->color = BLACK;
            singlerotationLL(parent);
            break;
        }
    }

    node->color = BLACK;
    return rbRoot(node);
}

/*
 * Delete 'key' from the Red-Black Tree and return the updated root.
 * Outputs the spliced-out node via '*deleted' so the caller can free it.
 */
RedBlackTreeNode* deleteRedBlack(RedBlackTreeNode* root, int key, RedBlackTreeNode** deleted) {
    RedBlackTreeNode* target = root;
    if(deleted != NULL) *deleted = NULL;

    /* Locate the target node containing 'key' */
    while(target != NULL && target->key != key) {
        if(key < target->key) {
            target = target->left;
        } else {
            target = target->right;
        }
    }
    if(target == NULL) return root; /* Key not found */

    /* If target has two children, swap its key with its in-order predecessor */
    RedBlackTreeNode* removed = target;
    if(target->left != NULL && target->right != NULL) {
        removed = target->left;
        while(removed->right != NULL) {
            removed = removed->right; /* Rightmost node in the left subtree */
        }
        RedBlackTreeNode* tept = target;
        int keyToDelete = tept->key;
        tept->key = removed->key;
        removed->key = keyToDelete;
    }

    /* Splice out 'removed', which now has at most one non-NULL child */
    RedBlackTreeNode* replacement = removed->left != NULL ? removed->left : removed->right;
    RedBlackTreeNode* parent = removed->parent;
    int removedWasLeft = parent != NULL && parent->left == removed;

    if(parent == NULL) {
        root = replacement; /* Removed node was the root */
    } else if(removedWasLeft) {
        parent->left = replacement;
```

```
    } else {
        parent->right = replacement;
    }
    if(replacement != NULL) replacement->parent = parent;

    /* If a BLACK node was removed, black-height is violated and requires fix-up */
    if(removed->color == BLACK) {
        if(replacement != NULL) {
            root = fixRedBlackAfterDelete(replacement);
        } else {
            /* Use a temporary stack sentinel to represent the double-black NULL leaf */
            RedBlackTreeNode temporary = {0, BLACK, NULL, NULL, parent};
            if(parent == NULL) {
                root = &temporary;
            } else if(removedWasLeft) {
                parent->left = &temporary;
            } else {
                parent->right = &temporary;
            }

            root = fixRedBlackAfterDelete(&temporary);
            /* Detach the temporary sentinel after rebalancing */
            if(temporary.parent != NULL) {
                if(temporary.parent->left == &temporary) temporary.parent->left = NULL;
                if(temporary.parent->right == &temporary) temporary.parent->right = NULL;
            }
            if(root == &temporary) root = NULL;
        }
    }

    /* Ensure the final tree root has NULL parent and BLACK color */
    if(root != NULL) {
        root = rbRoot(root);
        root->parent = NULL;
        root->color = BLACK;
    }
    removed->left = NULL;
    removed->right = NULL;
    removed->parent = NULL;
    if(deleted != NULL) *deleted = removed;
    return root;
}
```

A.5 Order-4 B+ Tree (`B_plus_tree.h`)

```
#ifdef _WIN32
    const char* const argv[] = {exe, "-Tpng", dotPath, "-o", pngPath, NULL};
    return _spawnvp(_P_WAIT, exe, argv) == 0 ? 0 : -1;
#else
    char command[1200]; snprintf(command, sizeof(command), "\"%s\" -Tpng \"%s\" -o \"%s\"", exe, dotPath, pngPath); return
        system(command) == 0 ? 0 : -1;
#endif
}
#endif

/*
 * B+ Tree node structure.
 * All actual data keys reside in leaf nodes (isLeaf == 1) linked via 'next'.
 * Internal nodes (isLeaf == 0) store separator keys where keys[i] = min(children[i+1]).
 */
typedef struct BPlusTreeNode BPlusTreeNode;

struct BPlusTreeNode {
    int isLeaf; /* 1 if leaf node, 0 if internal routing node */
    int numKeys; /* Number of keys currently stored in this node */
    int* keys; /* Dynamic array of keys (capacity = order) */
    BPlusTreeNode** children; /* Dynamic array of child pointers (capacity = order + 1) */
    BPlusTreeNode* parent; /* Pointer to the parent node */
    BPlusTreeNode* next; /* Pointer to the next leaf in sorted order */
};

/*
 * Wrapper structure representing a B+ Tree of a given branching order M.
```

```c
 */
typedef struct {
    int order; /* Maximum number of children per internal node (M = 4) */
    BPlusTreeNode* root; /* Pointer to the root node */
} BPlusTree;

/*
 * Allocate and initialize a new B+ Tree node (leaf or internal).
 */
static BPlusTreeNode* createBPlusNode(const BPlusTree* tree, int isLeaf) {
    BPlusTreeNode* node = (BPlusTreeNode*)malloc(sizeof(*node));
    if(node == NULL) return NULL;

    /* Allocate space for up to M keys and M + 1 children to accommodate temporary overflow */
    node->keys = (int*)malloc((size_t)tree->order * sizeof(*node->keys));
    node->children = (BPlusTreeNode**)calloc((size_t)tree->order + 1,
                                    sizeof(*node->children));
    if(node->keys == NULL || node->children == NULL) {
        free(node->keys);
        free(node->children);
        free(node);
        return NULL;
    }

    node->isLeaf = isLeaf;
    node->numKeys = 0;
    node->parent = NULL;
    node->next = NULL;
    return node;
}

/*
 * Free a single B+ Tree node and its internal arrays.
 */
static void freeBPlusNode(BPlusTreeNode* node) {
    if(node == NULL) return;
    free(node->keys);
    free(node->children);
    free(node);
}

/*
 * Return the smallest key in the subtree rooted at 'node' by walking down children[0].
 */
static int firstBPlusKey(BPlusTreeNode* node) {
    while(node != NULL && !node->isLeaf) node = node->children[0];
    return node->keys[0];
}

/*
 * Recompute separator keys of an internal node so keys[i] equals min(children[i+1]).
 */
static void refreshBPlusNode(BPlusTreeNode* node) {
    if(node == NULL || node->isLeaf) return;
    for(int i = 0; i < node->numKeys; ++i)
        node->keys[i] = firstBPlusKey(node->children[i + 1]);
}

/*
 * Refresh separator keys for 'node' and all of its ancestors up to the root.
 */
static void refreshBPlusAncestors(BPlusTreeNode* node) {
    while(node != NULL) {
        refreshBPlusNode(node);
        node = node->parent;
    }
}

/*
 * Descend from the root to locate the leaf node where 'key' belongs.
 */
static BPlusTreeNode* findBPlusLeaf(const BPlusTree* tree, int key) {
    BPlusTreeNode* node = tree->root;
    while(node != NULL && !node->isLeaf) {
        int index = 0;
```

```
        /* Advance child index while key is >= current separator key */
        while(index < node->numKeys && key >= node->keys[index]) ++index;
        node = node->children[index];
    }
    return node;
}

/*
 * Find the index of 'child' in 'parent->children[]'.
 */
static int indexOfBPlusChild(BPlusTreeNode* parent, BPlusTreeNode* child) {
    for(int i = 0; i <= parent->numKeys; ++i)
        if(parent->children[i] == child) return i;
    return -1;
}

static int insertBPlusChild(BPlusTree* tree, BPlusTreeNode* left,
                            BPlusTreeNode* right);

/*
 * Split an overflowing internal node with M + 1 children into two internal nodes.
 */
static int splitBPlusInternal(BPlusTree* tree, BPlusTreeNode* node) {
    BPlusTreeNode* right = createBPlusNode(tree, 0);
    if(right == NULL) return -1;

    /* Divide children evenly between the left node and the new right sibling */
    int totalChildren = node->numKeys + 1;
    int leftChildren = (totalChildren + 1) / 2;
    int rightChildren = totalChildren - leftChildren;
    right->numKeys = rightChildren - 1;

    /* Move the upper half of child pointers into 'right' */
    for(int i = 0; i < rightChildren; ++i) {
        right->children[i] = node->children[leftChildren + i];
        right->children[i]->parent = right;
        node->children[leftChildren + i] = NULL;
    }
    node->numKeys = leftChildren - 1;
    refreshBPlusNode(node);
    refreshBPlusNode(right);
    /* Attach the new right sibling into the parent node */
    return insertBPlusChild(tree, node, right);
}

/*
 * Insert 'right' as the child immediately following 'left' in their shared parent.
 * Creates a new root if 'left' was the root, or splits the parent if it overflows.
 */
static int insertBPlusChild(BPlusTree* tree, BPlusTreeNode* left,
                            BPlusTreeNode* right) {
    BPlusTreeNode* parent = left->parent;
    /* If the root split, allocate a new root node above 'left' and 'right' */
    if(parent == NULL) {
        BPlusTreeNode* root = createBPlusNode(tree, 0);
        if(root == NULL) return -1;
        root->numKeys = 1;
        root->children[0] = left;
        root->children[1] = right;
        left->parent = root;
        right->parent = root;
        refreshBPlusNode(root);
        tree->root = root;
        return 1;
    }

    /* Shift existing children right to insert 'right' at index + 1 */
    int index = indexOfBPlusChild(parent, left);
    for(int i = parent->numKeys + 1; i > index + 1; --i)
        parent->children[i] = parent->children[i - 1];
    parent->children[index + 1] = right;
    right->parent = parent;
    ++parent->numKeys;
    refreshBPlusNode(parent);

```

```
    /* Propagate split upward if parent now has M + 1 children (numKeys == M) */
    if(parent->numKeys >= tree->order)
        return splitBPlusInternal(tree, parent);
    refreshBPlusAncestors(parent->parent);
    return 1;
}

/*
 * Split an overflowing leaf node containing M keys into two leaves of ceil(M/2) and floor(M/2) keys.
 */
static int splitBPlusLeaf(BPlusTree* tree, BPlusTreeNode* leaf) {
    BPlusTreeNode* right = createBPlusNode(tree, 1);
    if(right == NULL) return -1;

    /* Move the upper half of the keys into the new right leaf */
    int leftKeys = (leaf->numKeys + 1) / 2;
    right->numKeys = leaf->numKeys - leftKeys;
    for(int i = 0; i < right->numKeys; ++i)
        right->keys[i] = leaf->keys[leftKeys + i];
    leaf->numKeys = leftKeys;

    /* Maintain the singly linked list of leaf nodes */
    right->next = leaf->next;
    leaf->next = right;
    return insertBPlusChild(tree, leaf, right);
}

/*
 * Create an empty B+ Tree of the specified branching order (order >= 3).
 */
BPlusTree* createBPlusTree(int order) {
    if(order < 3) return NULL;
    BPlusTree* tree = (BPlusTree*)malloc(sizeof(*tree));
    if(tree == NULL) return NULL;
    tree->order = order;
    tree->root = NULL;
    return tree;
}

/*
 * Search for 'key' in the B+ Tree; returns 1 if found, 0 otherwise.
 */
int searchBPlus(const BPlusTree* tree, int key) {
    if(tree == NULL) return 0;
    BPlusTreeNode* leaf = findBPlusLeaf(tree, key);
    if(leaf == NULL) return 0;
    for(int i = 0; i < leaf->numKeys; ++i)
        if(leaf->keys[i] == key) return 1;
    return 0;
}

/*
 * Insert 'key' into the B+ Tree.
 * Splits the target leaf (and any overflowing ancestors) when the leaf reaches M keys.
 */
int insertBPlus(BPlusTree* tree, int key) {
    if(tree == NULL || tree->order < 3) return -1;
    /* If the tree is empty, create the initial root leaf node */
    if(tree->root == NULL) {
        tree->root = createBPlusNode(tree, 1);
        if(tree->root == NULL) return -1;
        tree->root->keys[0] = key;
        tree->root->numKeys = 1;
        return 1;
    }

    /* Locate the target leaf and find the sorted insertion position */
    BPlusTreeNode* leaf = findBPlusLeaf(tree, key);
    int index = 0;
    while(index < leaf->numKeys && leaf->keys[index] < key) ++index;
    if(index < leaf->numKeys && leaf->keys[index] == key) return 0; /* Duplicate key */

    /* Shift larger keys right and insert 'key' */
    for(int i = leaf->numKeys; i > index; --i)
        leaf->keys[i] = leaf->keys[i - 1];
```

```
    leaf->keys[index] = key;
    ++leaf->numKeys;

    /* Split leaf if it reached M keys; otherwise refresh ancestor separators */
    if(leaf->numKeys >= tree->order) return splitBPlusLeaf(tree, leaf);
    refreshBPlusAncestors(leaf->parent);
    return 1;
}

/*
 * Remove child pointer at 'index' from 'parent' and update its separator keys.
 */
static void removeBPlusChild(BPlusTreeNode* parent, int index) {
    for(int i = index; i < parent->numKeys; ++i)
        parent->children[i] = parent->children[i + 1];
    parent->children[parent->numKeys] = NULL;
    --parent->numKeys;
    refreshBPlusNode(parent);
}

/*
 * Restore minimum occupancy at an internal node after a child removal.
 * Borrows a child from an adjacent sibling if possible, or merges with a sibling.
 */
static void rebalanceBPlusInternal(BPlusTree* tree, BPlusTreeNode* node) {
    /* If root has only one child left, collapse the root level */
    if(node == tree->root) {
        if(node->numKeys == 0) {
            tree->root = node->children[0];
            if(tree->root != NULL) tree->root->parent = NULL;
            freeBPlusNode(node);
        } else {
            refreshBPlusNode(node);
        }
        return;
    }

    /* Check whether the internal node still satisfies minimum key count ceil(M/2) - 1 */
    int minKeys = (tree->order + 1) / 2 - 1;
    if(node->numKeys >= minKeys) {
        refreshBPlusAncestors(node);
        return;
    }

    BPlusTreeNode* parent = node->parent;
    int index = indexOfBPlusChild(parent, node);
    BPlusTreeNode* left = index > 0 ? parent->children[index - 1] : NULL;
    BPlusTreeNode* right = index < parent->numKeys ? parent->children[index + 1] : NULL;

    /* Case 1a: Borrow the rightmost child from the left sibling */
    if(left != NULL && left->numKeys > minKeys) {
        for(int i = node->numKeys + 1; i > 0; --i)
            node->children[i] = node->children[i - 1];
        node->children[0] = left->children[left->numKeys];
        node->children[0]->parent = node;
        left->children[left->numKeys] = NULL;
        --left->numKeys;
        ++node->numKeys;
        refreshBPlusNode(left);
        refreshBPlusAncestors(node);
        return;
    }

    /* Case 1b: Borrow the leftmost child from the right sibling */
    if(right != NULL && right->numKeys > minKeys) {
        node->children[node->numKeys + 1] = right->children[0];
        node->children[node->numKeys + 1]->parent = node;
        ++node->numKeys;
        for(int i = 0; i < right->numKeys; ++i)
            right->children[i] = right->children[i + 1];
        right->children[right->numKeys] = NULL;
        --right->numKeys;
        refreshBPlusNode(right);
        refreshBPlusAncestors(node);
        return;
```

```
    }

    /* Case 2: Merge with left sibling (or right sibling) and propagate upward */
    if(left != NULL) {
        int first = left->numKeys + 1;
        for(int i = 0; i <= node->numKeys; ++i) {
            left->children[first + i] = node->children[i];
            node->children[i]->parent = left;
        }
        left->numKeys += node->numKeys + 1;
        removeBPlusChild(parent, index);
        refreshBPlusNode(left);
        freeBPlusNode(node);
    } else {
        int first = node->numKeys + 1;
        for(int i = 0; i <= right->numKeys; ++i) {
            node->children[first + i] = right->children[i];
            right->children[i]->parent = node;
        }
        node->numKeys += right->numKeys + 1;
        removeBPlusChild(parent, index + 1);
        refreshBPlusNode(node);
        freeBPlusNode(right);
    }
    rebalanceBPlusInternal(tree, parent);
}

/*
 * Delete 'key' from the B+ Tree.
 * Handles underflow in leaf nodes (< floor(M/2) keys) by borrowing or merging.
 */
int deleteBPlus(BPlusTree* tree, int key) {
    if(tree == NULL || tree->root == NULL) return 0;
    BPlusTreeNode* leaf = findBPlusLeaf(tree, key);
    int position = 0;
    while(position < leaf->numKeys && leaf->keys[position] < key) ++position;
    if(position == leaf->numKeys || leaf->keys[position] != key) return 0; /* Key absent */

    /* Remove the key from the leaf by shifting subsequent keys left */
    for(int i = position; i + 1 < leaf->numKeys; ++i)
        leaf->keys[i] = leaf->keys[i + 1];
    --leaf->numKeys;

    /* If the root itself is a leaf, allow 0..M-1 keys and free if empty */
    if(leaf == tree->root) {
        if(leaf->numKeys == 0) {
            freeBPlusNode(leaf);
            tree->root = NULL;
        }
        return 1;
    }

    /* If the leaf still has at least floor(M/2) keys, only refresh ancestor separators */
    int minKeys = tree->order / 2;
    if(leaf->numKeys >= minKeys) {
        refreshBPlusAncestors(leaf->parent);
        return 1;
    }

    BPlusTreeNode* parent = leaf->parent;
    int index = indexOfBPlusChild(parent, leaf);
    BPlusTreeNode* left = index > 0 ? parent->children[index - 1] : NULL;
    BPlusTreeNode* right = index < parent->numKeys ? parent->children[index + 1] : NULL;

    /* Borrow a key from the left sibling leaf if it has more than minKeys */
    if(left != NULL && left->numKeys > minKeys) {
        for(int i = leaf->numKeys; i > 0; --i) leaf->keys[i] = leaf->keys[i - 1];
        leaf->keys[0] = left->keys[left->numKeys - 1];
        --left->numKeys;
        ++leaf->numKeys;
        refreshBPlusAncestors(parent);
        return 1;
    }

    /* Borrow a key from the right sibling leaf if it has more than minKeys */
```

```
    if(right != NULL && right->numKeys > minKeys) {
        leaf->keys[leaf->numKeys++] = right->keys[0];
        for(int i = 0; i + 1 < right->numKeys; ++i) right->keys[i] = right->keys[i + 1];
        --right->numKeys;
        refreshBPlusAncestors(parent);
        return 1;
    }

    /* Neither sibling can lend a key: merge 'leaf' with an adjacent sibling */
    if(left != NULL) {
        for(int i = 0; i < leaf->numKeys; ++i)
            left->keys[left->numKeys + i] = leaf->keys[i];
        left->numKeys += leaf->numKeys;
        left->next = leaf->next;
        removeBPlusChild(parent, index);
        freeBPlusNode(leaf);
    } else {
        for(int i = 0; i < right->numKeys; ++i)
            leaf->keys[leaf->numKeys + i] = right->keys[i];
        leaf->numKeys += right->numKeys;
        leaf->next = right->next;
        removeBPlusChild(parent, index + 1);
        freeBPlusNode(right);
    }
    /* Rebalance the parent internal node after removing the merged child pointer */
    rebalanceBPlusInternal(tree, parent);
    return 1;
}

/*
```

A.6 Performance Benchmark Driver (`testbench_time.c`)

```
/*
 * File: testbench_time.c
 * Description: Automated performance benchmark for comparing Unbalanced BST,
 * AVL Tree, Splay Tree, Red-Black Tree, and Order-4 B+ Tree
 * under three insertion/deletion workloads.
 * Usage: testbench_time [startN] [endN] [step] [repetitions] [output.csv]
 */

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#ifdef _WIN32
#include <windows.h>
#endif
#include "binaray_search_tree.h"
#include "AVL_tree.h"
#include "splay_tree.h"
#include "red_black_tree.h"
#include "B_plus_tree.h"

/* Function pointer type and adapter entry for uniform benchmarking */
typedef void (*RunFunc)(const int*, const int*, int, double*, double*);
typedef struct { const char *name; RunFunc run; } TreeAdapter;

/*
 * High-resolution wall-clock timer returning elapsed time in seconds.
 */
static double nowSeconds(void) {
#ifdef _WIN32
    LARGE_INTEGER counter, frequency;
    QueryPerformanceFrequency(&frequency);
    QueryPerformanceCounter(&counter);
    return (double)counter.QuadPart / (double)frequency.QuadPart;
#else
    struct timespec t;
    timespec_get(&t, TIME_UTC);
    return (double)t.tv_sec + (double)t.tv_nsec / 1000000000.0;
#endif
}

```

```c
/*
 * Generate a uniform random permutation of {0, 1, ..., n - 1} using Fisher-Yates shuffle.
 */
static int *permutation(int n, unsigned seed) {
    int *v = (int*)malloc((size_t)n * sizeof(*v));
    if(!v) return NULL;
    for(int i = 0; i < n; i++) v[i] = i;
    srand(seed);
    for(int i = n - 1; i > 0; i--){
        int j = rand() % (i + 1), t = v[i];
        v[i] = v[j];
        v[j] = t;
    }
    return v;
}

/* Helper routines to free any remaining nodes after a test run */
static void freeBST(BSTreeNode*p){if(p){freeBST(p->Left);freeBST(p->Right);free(p);}}
static void freeAVL(AVLTreeNode*p){if(p){freeAVL(p->Left);freeAVL(p->Right);free(p);}}
static void freeSplay(SplayTreeNode*p){if(p){freeSplay(p->left);freeSplay(p->right);free(p);}}
static void freeRB(RedBlackTreeNode*p){if(p){freeRB(p->left);freeRB(p->right);free(p);}}

/*
 * Per-structure benchmark runners: measure total time for N insertions and N deletions.
 */
static void runBST(const int*i,const int*d,int n,double*ti,double*td){BSTreeNode*r=NULL;double a=nowSeconds();for(int k=0;k<n;k++)r=insertBST(r,i[k]);*ti=nowSeconds()-a;a=nowSeconds();for(int k=0;k<n;k++)r=deleteBST(r,d[k]);*td=nowSeconds()-a;freeBST(r);}
static void runAVL(const int*i,const int*d,int n,double*ti,double*td){AVLTreeNode*r=NULL;double a=nowSeconds();for(int k=0;k<n;k++)r=insertAVL(r,i[k]);*ti=nowSeconds()-a;a=nowSeconds();for(int k=0;k<n;k++)r=deleteAVL(r,d[k]);*td=nowSeconds()-a;freeAVL(r);}
static void runSplay(const int*i,const int*d,int n,double*ti,double*td){SplayTreeNode*r=NULL;double a=nowSeconds();for(int k=0;k<n;k++)r=insertSplay(r,i[k]);*ti=nowSeconds()-a;a=nowSeconds();for(int k=0;k<n;k++)r=deleteSplay(r,d[k]);*td=nowSeconds()-a;freeSplay(r);}
static void runRB(const int*i,const int*d,int n,double*ti,double*td){RedBlackTreeNode*r=NULL;double a=nowSeconds();for(int k=0;k<n;k++)r=insertRedBlack(r,i[k]);*ti=nowSeconds()-a;a=nowSeconds();for(int k=0;k<n;k++){RedBlackTreeNode*x=NULL;r=deleteRedBlack(r,d[k],&x);free(x);}*td=nowSeconds()-a;freeRB(r);}
static void runBPlus(const int*i,const int*d,int n,double*ti,double*td){BPlusTree*r=createBPlusTree(4);double a=nowSeconds();for(int k=0;k<n;k++)insertBPlus(r,i[k]);*ti=nowSeconds()-a;a=nowSeconds();for(int k=0;k<n;k++)deleteBPlus(r,d[k]);*td=nowSeconds()-a;destroyBPlusTree(r);}

/*
 * Run all five trees under a given workload scenario and record average timings to CSV.
 */
static void benchmark(FILE*out,const char*scenario,int n,int repeats,const int*ins,const int*del){
    TreeAdapter trees[]={{"BST",runBST},{"AVL",runAVL},{"Splay",runSplay},{"RedBlack",runRB},{"BPlus",runBPlus}};
    for(size_t t=0;t<sizeof(trees)/sizeof(trees[0]);t++){
        double it=0,dt=0,a,b;
        for(int r=0;r<repeats;r++){trees[t].run(ins,del,n,&a,&b);it+=a;dt+=b;}
        fprintf(out,"%s,%s,%d,%.9f,%.9f\n",scenario,trees[t].name,n,it/repeats,dt/repeats);
    }
}

/*
 * Main entry point: iterate over N = start..end and evaluate the three scenarios:
 * 1. ordered: insert 0..N-1, delete 0..N-1
 * 2. ordered_reverse: insert 0..N-1, delete N-1..0
 * 3. random: insert random permutation, delete random permutation
 */
int main(int argc,char**argv){
    int start=argc>1?atoi(argv[1]):1000, end=argc>2?atoi(argv[2]):50000;
    int step=argc>3?atoi(argv[3]):1000, repeats=argc>4?atoi(argv[4]):3;
    const char*path=argc>5?argv[5]:"data_set.csv";
    if(start<=0||end<start||step<=0||repeats<=0){fprintf(stderr,"Invalid range or repetitions.\n");return 1;}
    FILE*out=fopen(path,"w");if(!out){perror(path);return 1;}
    fprintf(out,"scenario,tree,n,insert_seconds,delete_seconds\n");
    for(int n=start;n<=end;n+=step){
        int*ordered= (int*)malloc((size_t)n*sizeof(*ordered));
        int*reversed= (int*)malloc((size_t)n*sizeof(*reversed));
        int*randomInsert=permutation(n,12345u+(unsigned)n);
        int*randomDelete=permutation(n,67890u+(unsigned)n);
        if(!ordered||!reversed||!randomInsert||!randomDelete){fprintf(stderr,"Out of memory at N=%d.\n",n);free(ordered);free(reversed);free(randomInsert);free(randomDelete);fclose(out);return 1;}
        for(int i=0;i<n;i++){
            ordered[i]=i; /* Increasing sequence 0, 1, ..., n - 1 */
```

```
105             reversed[i]=n-1-i; /* Decreasing sequence n - 1, ..., 1, 0 */
106         }
107         fprintf(stderr,"Testing N=%d/%d\n",n,end);
108         benchmark(out,"ordered",n,repeats,ordered,ordered);
109         benchmark(out,"ordered_reverse",n,repeats,ordered,reversed);
110         benchmark(out,"random",n,repeats,randomInsert,randomDelete);
111         free(ordered);free(reversed);free(randomInsert);free(randomDelete);
112         if(end-n<step) break; /* Avoid integer overflow at the upper bound */
113     }
114     fclose(out);printf("Benchmark written to %s\n",path);return 0;
115 }
```